Práctica 3: Calidad de la visualización – Checks

Presentado por Sara V. Guerrero Espinosa
10 de marzo de 2026

Índice:

Preparación del entorno..... 1
Matriz de calidad..... 5
Materialización del pipeline y resultados de las validaciones..... 9
Control de versiones e integración continua..... 11

En la presente práctica se abordó la incorporación de mecanismos de validación de calidad dentro de un pipeline de datos orientado a la visualización, utilizando Dagster como herramienta de orquestación. El objetivo principal consistió en diseñar e implementar un conjunto de verificaciones sistemáticas (checks) que permitieran garantizar la integridad, coherencia y consistencia de los datos a lo largo de todas las etapas del proceso, desde la carga hasta la generación de las visualizaciones. Asimismo, se buscó reforzar el enfoque DataOps aplicado en la práctica anterior, incorporando controles automatizados que aseguren la fiabilidad de los resultados y faciliten la detección temprana de posibles problemas de calidad.

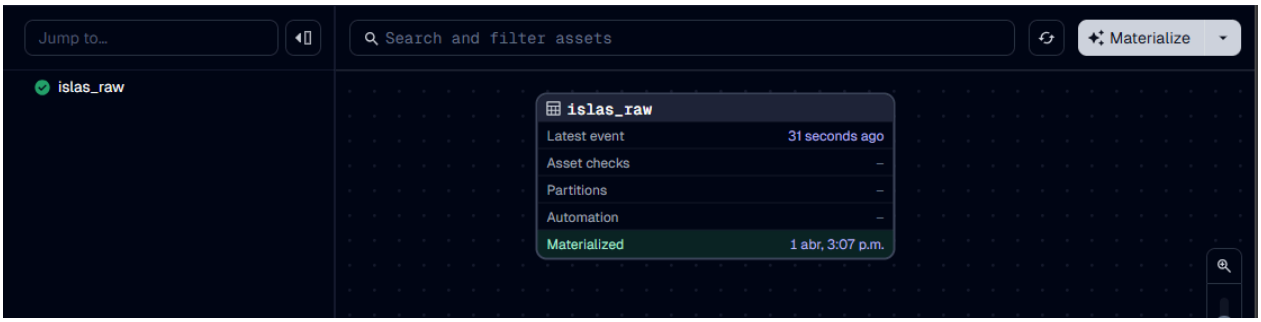
Todos los ficheros de trabajo, incluídos los datos utilizados, el código de Python, las visualizaciones generadas y este mismo informe se encuentran subidos al repositorio de Github dispuesto en el siguiente link: [Práctica 03 Visualización - GitHub Sara Guerrero]

A continuación se detalla la solución de los diferentes apartados de la práctica:

Preparación del entorno

Dentro del entorno de Dagster, los checks constituyen mecanismos de validación asociados a los assets que permiten evaluar de forma automática la calidad de los datos generados en cada etapa del pipeline. Su función consiste en verificar que determinadas condiciones se cumplan, como la ausencia de valores nulos, la coherencia de formatos o la validez de rangos, entre otras. Estos controles resultan fundamentales en un contexto DataOps, ya que permiten detectar errores de forma temprana, evitar la propagación de inconsistencias a etapas posteriores y garantizar que las visualizaciones finales se basen en información fiable. De este modo, los checks no solo validan el estado técnico de los datos, sino que también contribuyen a asegurar su correcta interpretación desde el punto de vista analítico y visual.

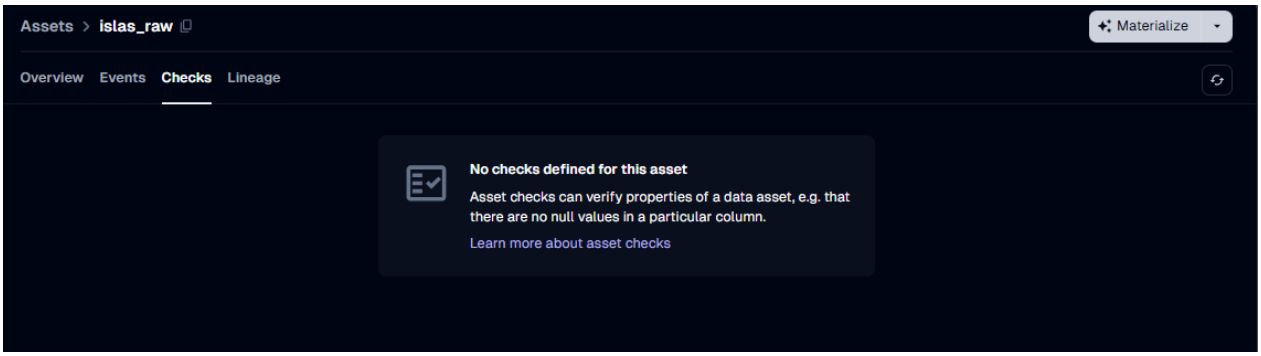
Antes de comenzar el desarrollo del pipeline principal, se realizó una prueba inicial utilizando el fichero *test_checks.py* con el objetivo de comprender el funcionamiento de los checks en Dagster. Este script consistía en la carga de un archivo CSV y la definición de un mecanismo de validación asociado.



El asset islas_raw se encargaba de cargar el dataset original y, de forma intencionada, añadía una fila con una inconsistencia en la variable categórica Isla, simulando un problema de calidad de datos. Sobre este asset se definió un asset check orientado a verificar la correcta estandarización de dicha variable. Para ello, se comparó el número de categorías únicas antes y después de aplicar una normalización básica basada en la capitalización de los valores. Si ambas cantidades diferían,

se interpretaba como evidencia de inconsistencias que podrían derivar en la duplicación de categorías dentro de las visualizaciones.

Al ejecutar el script desde *test_checks.py*, el asset se materializó correctamente; sin embargo, el check no fue ejecutado. Este comportamiento se debió a la ausencia del bloque Definitions, el cual resulta necesario para que Dagster registre explícitamente tanto los assets como sus validaciones. En consecuencia, aunque el check se encontraba correctamente definido a nivel de código, no era visible ni ejecutable dentro del entorno de Dagster.



A continuación, se procedió a revisar el fichero *definitions.py* proporcionado en la práctica, el cual constituye el punto de configuración central del proyecto en Dagster.

El enunciado planteaba analizar la estructura de este fichero e identificar qué assets y qué checks formaban parte del manifiesto del pipeline. En este contexto, el archivo definitions.py se encarga de declarar explícitamente los componentes que Dagster debe registrar y ejecutar. En él se importa el módulo test_checks, que contiene tanto los assets como sus correspondientes validaciones, y se construye un objeto Definitions que agrupa todos estos elementos.

Los assets se cargan de forma dinámica mediante la función load_assets_from_modules, lo que permite incorporar automáticamente todas las funciones decoradas con @asset definidas en el módulo. De manera análoga, los checks se integran a través de load_asset_checks_from_modules, incluyendo todas las funciones decoradas con @asset_check. Esta estructura permite a Dagster descubrir, organizar y ejecutar tanto la generación de datos como sus verificaciones de calidad dentro de un mismo manifiesto, manteniendo un enfoque modular y escalable.

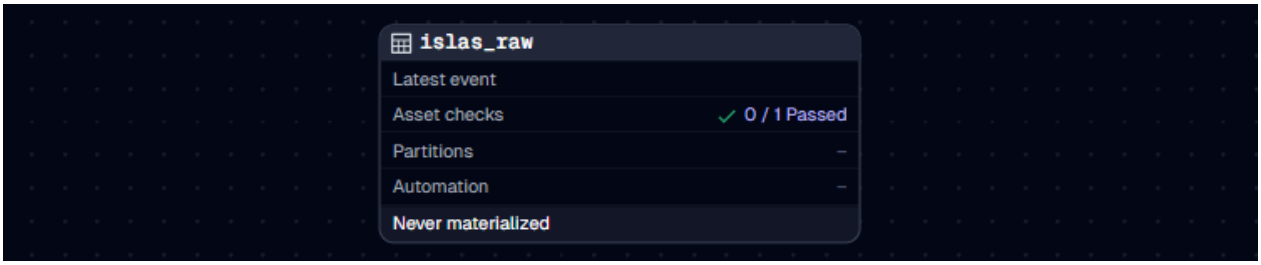
Practica_03 > definitions.py > ...

```
from dagster import Definitions, load_assets_from_modules, load_asset_checks_from_modules
# Supongamos que tu archivo se llama proyecto_islas.py
from scripts import test_checks

defs = Definitions(
    assets=load_assets_from_modules([test_checks]),
    # ¡AQUÍ ESTÁ LA CLAVE! Debes añadir el check aquí:
    asset_checks=load_asset_checks_from_modules([test_checks])
)
```

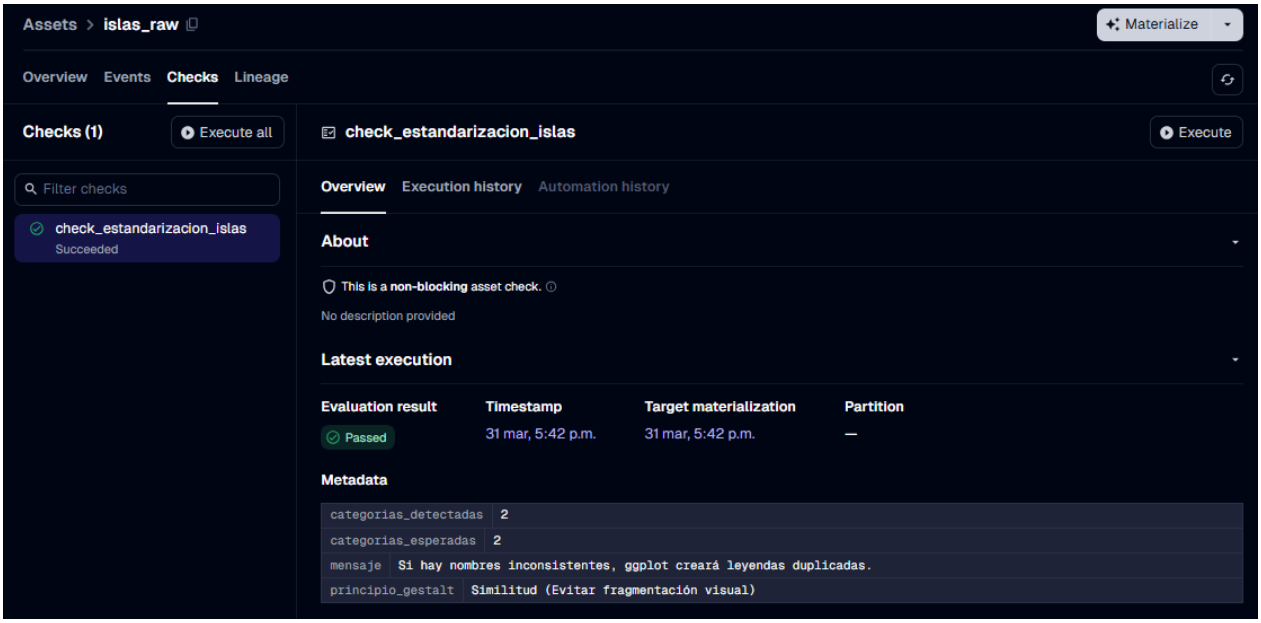
En cuanto al contenido del manifiesto, se identificó el asset islas_raw, definido en el módulo test_checks e incorporado mediante el mecanismo de carga dinámica. Asimismo, se incluyó el

check check_estandarizacion_islas, asociado a dicho asset y registrado a través de la función correspondiente para asset checks.



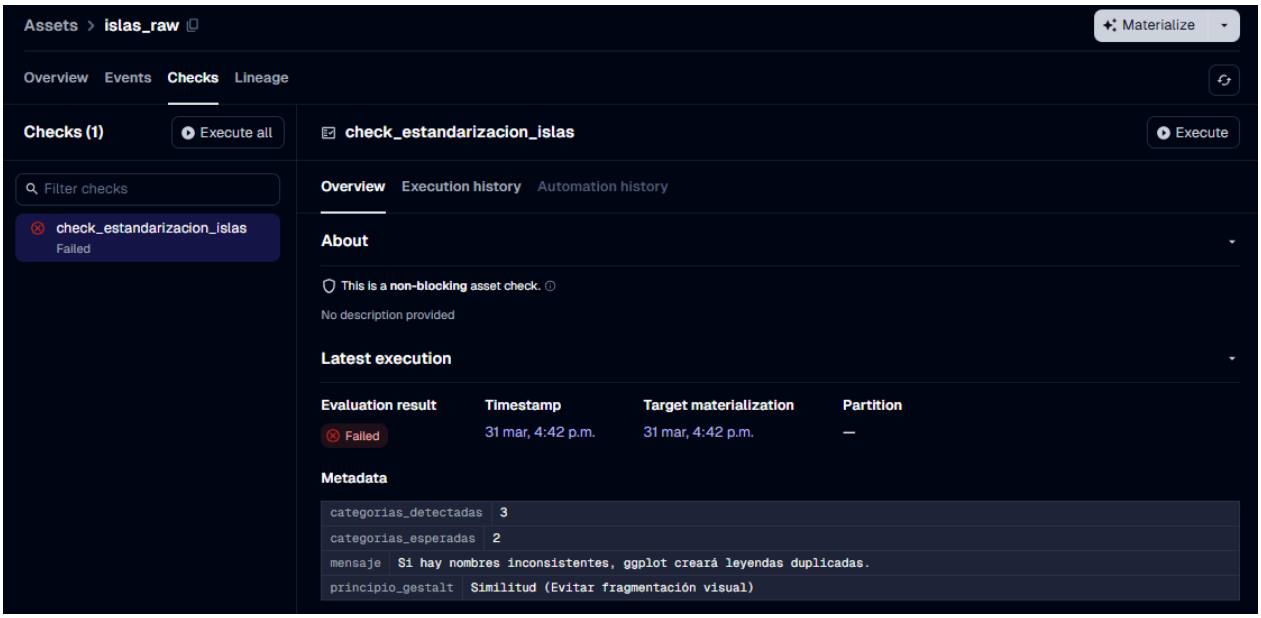
Una vez cargado el proyecto desde definitions.py, tanto los assets como los checks quedaron correctamente definidos dentro del entorno de Dagster. Al materializar el pipeline, se comprobó que la ejecución se realizaba de forma satisfactoria y que los checks eran evaluados correctamente, obteniendo un resultado positivo. En este caso, el número de categorías detectadas coincidía con el esperado, lo que indicaba la ausencia de inconsistencias en la variable analizada.

Los metadatos asociados reflejaban esta situación, y se destacaba la importancia de evitar la fragmentación de categorías desde el punto de vista del principio de similitud en la percepción visual.



Posteriormente, se introdujo de forma deliberada una fila inconsistente en el dataset, simulando un problema de calidad de datos. Tras recargar las definiciones y materializar nuevamente el pipeline, se observó que el asset continuaba ejecutándose correctamente, pero el check no fue superado.

En este caso, el número de categorías detectadas aumentó respecto al esperado, evidenciando la presencia de valores inconsistentes en la variable categórica. Esta situación confirma la capacidad de los checks para detectar problemas que, de no corregirse, podrían derivar en la duplicación de categorías en las visualizaciones y, por tanto, en una interpretación errónea de los datos.



Una vez comprendido el funcionamiento de los checks y verificado su comportamiento ante datos consistentes e inconsistentes, se dio por finalizada esta fase de prueba, permitiendo proceder con el desarrollo del pipeline principal utilizando los assets y datasets de interés.

Matriz de calidad

Una vez configurado el entorno de Dagster, se procedió al proyecto de distribución de renta en las islas. Para este se debían incorporar los checks correspondientes dentro del pipeline principal, desarrollado en la práctica anterior, con el fin de extender el control de calidad a lo largo de todo el flujo de datos.

Como punto de partida, se retomó la estructura del pipeline previamente implementado. Los datasets utilizados fueron los siguientes. En primer lugar, distribucion-renta-canarias, que contiene información sobre la distribución de la renta bruta media por persona según fuente de ingresos en Canarias, desagregada por islas y municipios, y con cobertura temporal entre 2015 y 2023. En segundo lugar, codislas, que incluye la relación de municipios de la Comunidad Autónoma de Canarias junto con su isla de pertenencia y su codificación territorial. Finalmente, nivelestudios, que recoge información sobre la población residente en municipios de 500 habitantes o más, desagregada por sexo, nacionalidad, nivel educativo y año (2021–2023), permitiendo calcular la proporción de población con educación superior en cada municipio.

A partir de estos datos, el pipeline se estructuró en cuatro fases principales: carga, limpieza, transformación/integración y visualización. Se definieron un total de doce assets. Las dependencias entre estos assets quedaron representadas en la vista gráfica del pipeline dentro de Dagster, donde se observó la relación jerárquica entre las distintas etapas del proceso y el flujo de datos desde su origen hasta la generación de las visualizaciones.

Con el objetivo de garantizar la calidad de los datos en cada una de estas fases, se diseñarían un conjunto de verificaciones sistemáticas (checks) alineadas con las etapas del pipeline.

En la siguiente tabla se presenta la planificación de los checks implementados, especificando los assets sobre los que se aplican, la lógica empleada para su evaluación y su relación con los principios de diseño visual. Esta matriz de calidad constituye el marco de referencia para el control sistemático de la calidad a lo largo del pipeline.

Etapa	Nombre del Check	Descripción Técnica	Assets	Cómo programarlo (Lógica)	Relación con el Diseño / Gestalt	Información para Metadata
Carga (Raw)	check_raw_no_vacio	Verifica que el dataset bruto se haya cargado correctamente y contenga registros. Aplicado a: raw_renta, raw_nombres, raw_estudios.	raw_renta, raw_nombres, raw_estudios	filas, columnas = df.shape passed = filas > 0 and columnas > 0	Figura-Fondo: Sin datos no hay visualización interpretable. Sin tabla de apoyo no se pueden vincular territorios e islas.	filas, columnas.
Limpieza (Clean)	check_sin_nulos_clave	Verifica que el dataset limpio no contenga valores nulos en las variables esenciales del análisis.	df_renta, df_nombres, df_estudios	nulos = df[columnas_clave].isna().sum().sum() passed = nulos == 0	Buena Continuidad: Los cortes en variables clave rompen el análisis. Sin claves completas el merge pierde correspondencias.	nulos_detectados, columnas_revisadas, columnas_afectadas
Limpieza (Clean)	check_sin_duplicados	Verifica que el dataset limpio de renta no contenga registros duplicados. Aplicado a: df_renta.	df_renta	duplicados = int(df_renta.duplicated().sum()) passed = duplicados == 0	Similitud: Los duplicados alteran el peso visual de las categorías.	duplicados_detectados, filas_afectadas
Limpieza (Clean)	check_porcentajes_validos	Verifica que los valores porcentuales estén expresados en una escala válida. Tal que: df_renta (escala 0-100); df_estudios (escala 0-1).	df_renta, df_estudios	fuera_rango = ((df[col] < min_val) | (df[col] > max_val)).sum() passed = fuera_rango == 0	Proporción: Una escala inválida distorsiona la comparación visual. Los porcentajes deben respetar su escala.	valores_fuera_de_rango, rango_valores
Limpieza	check_df_renta_est	Verifica que los nombres	df_renta	originales =	Similitud:	territorios_detecta

(Clean)	andarizacion_texto	de territorios y medidas estén estandarizados para evitar fragmentación de categorías en las visualizaciones.		df[col].nunique() normalizadas = df[col].str.strip().str.title().nunique() passed = originales == normalizadas	Nombres inconsistentes crearían categorías duplicadas.	dos, territorios_normalizados, medidas_detectadas, medidas_normalizadas.
Limpieza (Clean)	check_df_nombres_estandarizacion_islas	Verifica que los nombres de isla estén estandarizados para evitar categorías duplicadas en el análisis territorial.	df_nombres	originales = df_nombres["Isla"].nunique() normalizadas = df_nombres["Isla"].astype(str).str.strip().str.title().nunique() passed = originales == normalizadas	Similitud: Evitar fragmentación visual.	categorias_detectadas, categorias_normalizadas
Limpieza (Clean)	check_codigo_unico	Verifica que cada código territorial identifique de forma única a un municipio o territorio.	df_nombres	duplicados = df['Código territorial'].duplicated().sum() passed = duplicados == 0	Conectividad: Cada territorio debe enlazar con una única referencia.	codigos_duplicados, valores_duplicados
Limpieza (Clean)	check_estudios_suma_por_municipio_ano	Verifica que la distribución de niveles educativos por municipio y año cierre aproximadamente al 100%.	df_estudios	suma = df.groupby(['Código territorial', 'Año'])['Porcentaje (%)'].sum().round(4) inconsistentes = (suma.sub(1).abs() > 0.01).sum() passed = inconsistentes == 0	Cierre: La distribución educativa debe completar el 100%.	grupos_inconsistentes, grupos_revisados.
Transf. (Curated)	check_renta_isla_sin_islas_nulas	Verifica que el cruce territorial haya asignado una isla a cada registro del dataset de renta	df_renta_isla	nulos = df['Isla'].isna().sum() passed = nulos == 0	Conectividad: Cada observación debe quedar ligada a una isla.	nulos_en_isla, filas_afectadas, territorios_afcetados

		enriquecido.			El merge no debe dejar territorios sin isla.	
Transf. (Curated)	check_renta_isla_tipos_validos	Verifica que la clasificación territorial resultante sólo contenga los tipos previstos por el pipeline (Municipio, Isla, Provincia, Comunidad).	df_renta_isla	esperados = {'Municipio', 'Isla', 'Provincia', 'Comunidad'} detectados = set(df['Tipo'].dropna().unique()) passed = len(detectados - esperados) == 0	Similitud: La clasificación territorial debe ser coherente y no introducir tipos inesperados.	tipos_detectados, tipos_invalidos.
Transf. (Curated)	check_renta_edu_cobertura_merge	Verifica que el cruce entre renta y educación conserve una cobertura suficiente de municipios con dato educativo disponible (umbral >= 90%).	df_renta_edu	municipios = df[df['Tipo'] == 'Municipio'] cobertura = municipios['Educación superior (%)'].notna().mean() * 100 passed = cobertura >= 90	Conectividad: El cruce debe conservar suficiente información compartida. Sin cobertura no hay análisis válido.	filas_municipales, filas_con_educacion, cobertura_pct.
Transf. (Curated)	check_renta_edu_porcentajes_validos	Verifica que el porcentaje de educación superior resultante del cruce esté expresado en una escala válida entre 0 y 100.	df_renta_edu	serie = df['Educación superior (%)'].dropna() fuera_rango = ((serie < 0) | (serie > 100)).sum() passed = fuera_rango == 0	Proporción: Las variables cuantitativas deben compartir escalas válidas.	valores_fuera_de_rango, rango_valores
Visualiz. (Asset)	check_grafico_exportado	Verifica que cada gráfico se haya exportado correctamente como archivo PNG con contenido no vacío.	graph_01, graph_02, graph_03, graph_04	path = VIS_DIR / nombre_archivo passed = path.exists() and path.stat().st_size > 0	Figura-Fondo: Sin exportación no hay salida visual evaluable.	archivo, tamano_bytes.

Además de los metadatos especificados en la tabla, en la implementación todos los checks incorporaron una descripción, el principio de Gestalt asociado y un mensaje interpretativo dirigido al usuario. Esta información complementaria permitió no solo validar técnicamente los datos, sino también contextualizar cada verificación desde el punto de vista del diseño visual.

Como se puede apreciar, los checks de mayor relevancia se concentraron en las etapas de limpieza y transformación, dado que en ellas se producen las principales modificaciones sobre los datos. En particular, adquirieron especial importancia las verificaciones relacionadas con la estandarización de categorías y la coherencia de las escalas, debido a su impacto directo en la correcta representación visual y en la interpretación de los resultados.

Por otra parte, la inclusión de checks en la fase de visualización permiten asegurar que los resultados del pipeline no solo sean correctos desde una perspectiva técnica, sino también que estén correctamente materializados y disponibles para su análisis.

A partir de la definición de esta matriz de calidad, se procedió a la implementación de los checks en el fichero *lab_renta_checks.py*. En total se implementaron 19 checks para los 12 assets.

```
Practica_03 > lab_renta_checks.py > check_df_renta_isla_sin_islas_nulas
import os
from pathlib import Path
import pandas as pd
from dagster import asset_check, AssetCheckResult, MetadataValue

from Practica_03 import lab_renta_assets


# ==============================================================
# CAPA 1: CARGA
# ==============================================================

@asset_check(
    asset=lab_renta_assets.raw_renta,
    description="Verifica que el dataset bruto de renta se haya cargado correctamente y contenga registros."
)
def check_raw_renta_no_vacio(raw_renta: pd.DataFrame) -> AssetCheckResult:
    filas, columnas = raw_renta.shape
    passed = filas > 0 and columnas > 0

    return AssetCheckResult(
        passed=passed,
        metadata={
            "filas": MetadataValue.int(filas),
            "columnas": MetadataValue.int(columnas),
            "principio_gestalt": "Figura-fondo (Sin datos no hay visualización interpretable)",
            "mensaje": "El dataset base no puede estar vacío."
```

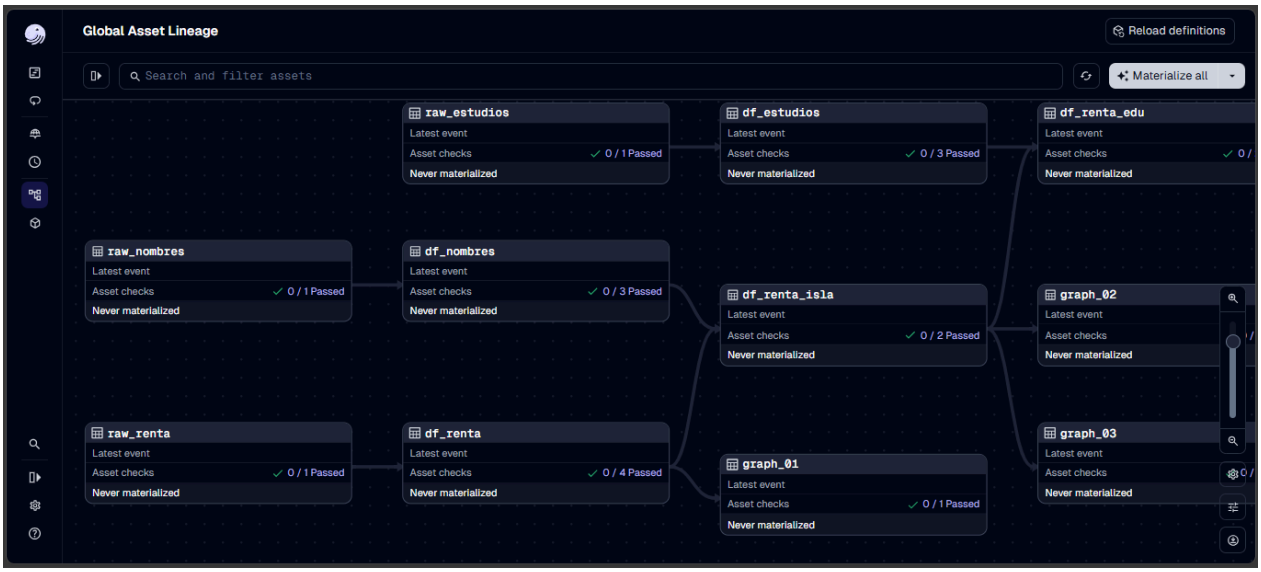
Materialización del pipeline y resultados de las validaciones

En primer lugar, procedió a ajustar el fichero *definitions.py* con el objetivo de integrar correctamente los componentes del pipeline. En particular, se configuró la carga de los assets desde el módulo *lab_renta_assets.py* y de los checks desde *lab_renta_checks.py*, asegurando que ambos quedaran registrados dentro del entorno de Dagster.

```
Practica_03 > definitions.py > ...
You, 27 seconds ago | 2 authors (unknown and one other)
from dagster import Definitions, load_assets_from_modules, load_asset_checks_from_modules
from Practica_03 import lab_renta_assets
from Practica_03 import lab_renta_checks

defs = Definitions(
    assets=load_assets_from_modules([lab_renta_assets]),
    asset_checks=load_asset_checks_from_modules([lab_renta_checks])
)
```

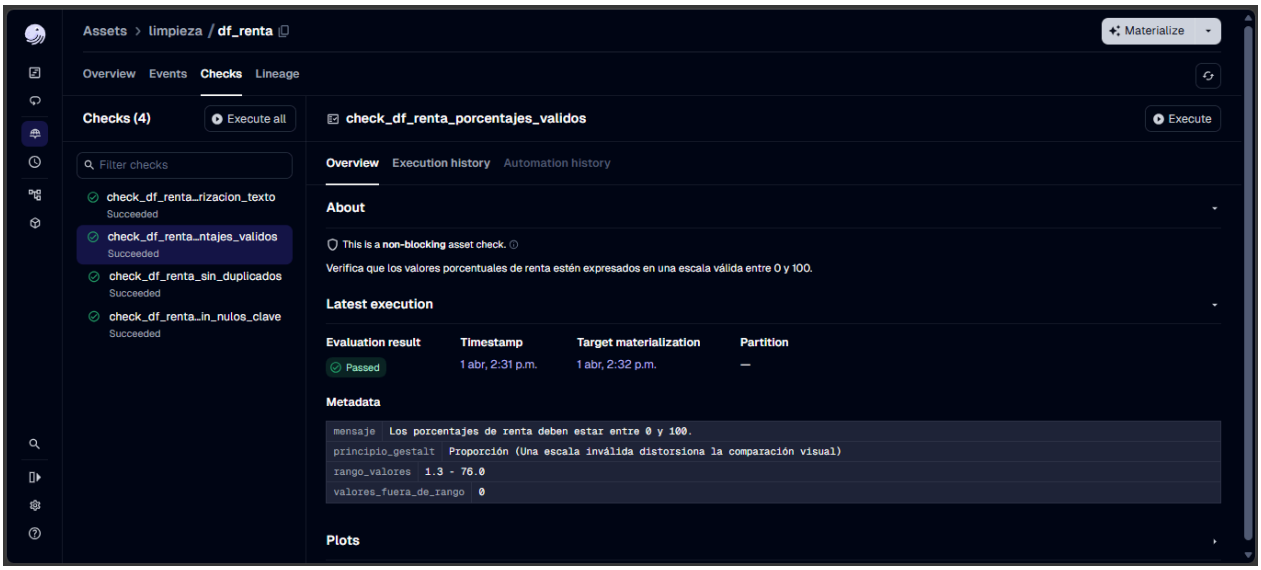
Una vez realizada esta configuración, se cargó el pipeline en Dagster. Se comprobó que tanto los assets como los checks fueron reconocidos correctamente, apareciendo de forma adecuada en la interfaz del sistema.



Posteriormente, se ejecutó la materialización completa del pipeline. La ejecución se llevó a cabo de manera satisfactoria y se completaron todas las etapas definidas.

Como resultado de la materialización, se observó que la mayoría de los checks fueron aprobados. No obstante, uno de ellos no superó la validación, lo cual resultaba esperado, ya que había sido diseñado deliberadamente para verificar el correcto funcionamiento del sistema de checks conforme a lo indicado en el enunciado de la práctica.

Finalmente, se realizó una revisión detallada de los checks asociados a cada asset mediante la pestaña correspondiente en Dagster. Esta inspección permite analizar individualmente cada validación y prestar especial atención a los metadatos generados, lo que facilita la interpretación de los resultados y la comprensión del estado de calidad de los datos en cada etapa del pipeline.



Con la correcta integración y validación de los checks, el pipeline quedó completamente operativo y preparado para su ejecución de forma controlada. La revisión de los resultados permitió comprobar el comportamiento esperado de las verificaciones, así como interpretar de manera detallada la información proporcionada en cada caso. De este modo, se completó el desarrollo de la práctica, dejando establecido un flujo de trabajo estructurado y validado conforme a los requisitos planteados.

Control de versiones e integración continua

El proyecto se gestionó mediante Git como sistema de control de versiones, lo que permitió registrar de forma estructurada cada modificación realizada en el pipeline y en las visualizaciones. El repositorio fue publicado en GitHub, asegurando un respaldo remoto del trabajo y facilitando el seguimiento del historial de cambios.

En el marco de DataOps, el uso de Git y GitHub resultó fundamental para garantizar la trazabilidad, la reproducibilidad y la integridad del proceso. Cada actualización del código quedó documentada mediante commits, lo que permitió validar progresivamente los avances y mantener una evolución ordenada del proyecto. De este modo, la orquestación con Dagster se complementa con buenas prácticas de control y gobernanza del desarrollo.